

Quantum Fourier Transform

Lorenzo Pasini

Anno Accademico 2025/2026

6 maggio 2026

Indice

1	Introduzione	4
1.1	Trasformata di Fourier continua	4
1.2	La Trasformata di Fourier Discreta	4
1.3	Cos'è la QFT	5
1.4	Formulazione della QFT in termini binari	6
1.5	Rappresentazione della QFT in forma matriciale	6
1.6	Interpretazione fisica: sovrapposizione e rotazione	7
1.6.1	Sovrapposizione di tutti gli stati	7
1.6.2	Conteggio nella base di Fourier	7
1.7	Implementazione circuitale	9
1.8	Vantaggi e Limiti Computazionali	9
2	Piattaforma	11
2.1	Accesso alla piattaforma IBM Quantum	11
2.2	Ambiente di sviluppo e strumenti utilizzati	11
2.3	Librerie e framework	11
2.4	Linguaggio di programmazione	12
3	Implementazione	13
3.1	Trasformata di Fourier Quantistica	13
3.1.1	Struttura dell'algoritmo	13
3.1.2	Pseudocodice	13
3.1.3	Implementazione in Python	13
3.2	Antitrasformata di Fourier Quantistica	14
3.2.1	Struttura dell'algoritmo	14
3.2.2	Pseudocodice	14
3.2.3	Implementazione in Python	15
3.3	Trasformata di Fourier Quantistica Approssimata	15
3.3.1	Struttura dell'algoritmo	15
3.3.2	Pseudocodice	16
3.3.3	Implementazione in Python	16
4	Esperimenti e Risultati	17
4.1	Test di correttezza	18
4.2	Trasformata di Fourier su Simulatore Aer	19
4.2.1	Esecuzione della QFT	19
4.3	Comparazione tra simulazione ideale e rumorosa	20
4.3.1	Analisi dei risultati	20
4.4	Libreria vs implementazione custom	21
4.4.1	Analisi dei risultati	21
4.5	Approssimata vs ideale	21

4.5.1	Analisi dei risultati	21
4.6	Esecuzione su QPU	22
4.6.1	Analisi dei risultati	22
4.7	Confronto sinottico degli errori	22
4.8	Analisi delle Performance: Tempi di Esecuzione e Risorse Hardware	22
4.8.1	Simulazione su CPU: Il Limite della Complessità Esponenziale	23
4.8.2	Esecuzione su QPU: Overhead Infrastrutturale	23
4.8.3	Confronto e Considerazioni Finali	23
5	Conclusioni	25
6	Riferimenti e Risorse Online	26

Capitolo 1

Introduzione

1.1 Trasformata di Fourier continua

La **trasformata di Fourier continua** è uno strumento matematico fondamentale che consente di rappresentare una funzione nel dominio delle frequenze, scomponendola in una combinazione continua di esponenziali complessi. Essa permette di analizzare il contenuto in frequenza di un segnale nel dominio continuo del tempo, ed è ampiamente utilizzata in fisica, ingegneria e teoria dei segnali.

Formalmente, la trasformata di Fourier di una funzione integrabile $f(t)$ è definita come:

$$F(\xi) = \int_{-\infty}^{+\infty} f(t) e^{-i\xi t} dt$$

dove:

- $f(t)$ è la funzione nel dominio del tempo;
- $F(\xi)$ è la sua rappresentazione nel dominio delle frequenze;
- ξ è la frequenza in [Hz].

La trasformazione inversa, detta **antitrasformata di Fourier**, consente di ricostruire il segnale originale a partire dalla sua rappresentazione in frequenza:

$$f(t) = \int_{-\infty}^{+\infty} F(\omega) e^{i\xi t} d\xi$$

La trasformata di Fourier costituisce il punto di partenza teorico per la **trasformata discreta di Fourier (DFT)** e, successivamente, per la **Quantum Fourier Transform (QFT)**.

1.2 La Trasformata di Fourier Discreta

Per applicazioni numeriche e computazionali, è necessario passare ad una rappresentazione discreta. Attraverso un processo di campionamento e discretizzazione, si ottiene la **Trasformata Discreta di Fourier (DFT)**.

Definizione 1.1: DFT - Discrete Fourier Transform

La DFT è definita come:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi \frac{kn}{N}}, \quad k = 0, \dots, N-1$$

dove:

- $x[n]$ rappresenta il segnale nel dominio temporale discreto;
- $X[k]$ rappresenta il segnale nel dominio delle frequenze;
- N è il numero di campioni.

L'interpretazione della DFT è quindi quella di una trasformazione tra il dominio del tempo e il dominio delle frequenze per segnali discreti e finiti.

Possiamo ottenere l'inversa, proprio come nel continuo, con l'anti-trasformata:

Definizione 1.2: IDFT - Inverse Discrete Fourier Transform

L'antitrasformata discreta di Fourier (IDFT) è definita come:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{i2\pi \frac{kn}{N}}, \quad n = 0, \dots, N-1$$

Essa consente di ricostruire il segnale originale a partire dalla sua rappresentazione nel dominio delle frequenze.

1.3 Cos'è la QFT

La **Quantum Fourier Transform (QFT)** può essere vista come la versione quantistica della trasformata discreta di Fourier, applicata agli stati di un sistema quantistico. Essa sfrutta i **principi di sovrapposizione** e **interferenza quantistica** per rappresentare l'informazione in termini di ampiezze e fasi.

Definizione 1.3: QFT - Quantum Fourier Transform

Formalmente, per uno stato base $|x\rangle$ con $x \in \{0, \dots, 2^n - 1\}$, la QFT è definita come:

$$\text{QFT}(|x\rangle) = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i2\pi \frac{xk}{N}} |k\rangle$$

dove $N = 2^n$ rappresenta la dimensione dello spazio di Hilbert.

Analogamente alla trasformata discreta di Fourier (DFT), anche la QFT ammette un'inversa, detta **Inverse Quantum Fourier Transform (IQFT)**, che consente di recuperare lo stato originale a partire dalla sua trasformazione nel dominio delle fasi.

Definizione 1.4: IQFT - Inverse Quantum Fourier Transform

L'IQFT è definita come l'operazione inversa della QFT:

$$\text{IQFT} = \text{QFT}^{-1}$$

e agisce sugli stati come:

$$|x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{-i2\pi \frac{xk}{N}} |k\rangle$$

1.4 Formulazione della QFT in termini binari

Vediamo una formulazione equivalente della trasformata di Fourier quantistica utilizzando la notazione binaria dei qubit. Questa rappresenta un passaggio intermedio utile per collegare la definizione matematica dell'operatore alla sua implementazione circuitale.

Un intero $x \in \{0, \dots, N-1\}$ può essere rappresentato in base 2 come:

$$x = x_{n-1}x_{n-2} \dots x_1x_0,$$

oppure in forma posizionale:

$$x = \sum_{j=0}^{n-1} x_j 2^j.$$

Dividendo per $N = 2^n$, si ottiene la rappresentazione frazionaria:

$$\frac{x}{N} = 0.x_{n-1}x_{n-2} \dots x_0 = \sum_{j=0}^{n-1} x_j 2^{j-n}.$$

Analogamente, anche k può essere rappresentato in forma binaria:

$$k = \sum_{j=0}^{n-1} k_j 2^j.$$

Il termine esponenziale della trasformata di Fourier discreta può quindi essere riscritto come:

$$e^{2\pi i \frac{xk}{N}}.$$

Sostituendo le espansioni binarie di x e k , e utilizzando la linearità dell'esponente, si ottiene una fattorizzazione in prodotti di fasi locali sui singoli bit.

In particolare, la trasformata di Fourier discreta può essere espressa nella forma:

$$\tilde{x}_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \omega_N^{jk},$$

dove:

$$\omega_N = e^{\frac{2\pi i}{N}}, \quad \omega_N^{jk} = e^{\frac{2\pi i}{N}jk}.$$

La funzione $e^{\frac{2\pi i}{N}jk}$ è periodica in entrambe le variabili, con periodo dipendente da $\frac{N}{k}$. Questo implica che la trasformata di Fourier può essere interpretata come una decomposizione della sequenza discreta $\{x_j\}$ in una combinazione lineare di componenti periodiche, ciascuna caratterizzata da una specifica frequenza.

In altre parole, la QFT riscrive lo stato computazionale come sovrapposizione di modi oscillatori discreti, permettendo di estrarre informazioni globali sulla periodicità della funzione rappresentata dal registro quantistico.

1.5 Rappresentazione della QFT in forma matriciale

Sia $\omega = e^{i\frac{2\pi}{N}}$ la radice N -esima dell'unità. La matrice della QFT di dimensione $N \times N$ è definita dall'elemento di matrice:

$$(\text{QFT})_{jk} = \frac{1}{\sqrt{N}} \omega^{jk}$$

Esplicitamente, la matrice assume la forma di una matrice di Vandermonde simmetrica:

$$\text{QFT} = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \cdots & \omega^{(N-1)(N-1)} \end{bmatrix}$$

Si noti che QFT_N è un'operazione **unitaria**, ovvero $\text{QFT}_N^\dagger \times \text{QFT}_N = I$, il che garantisce la reversibilità del processo (Inverse QFT).

1.6 Interpretazione fisica: sovrapposizione e rotazione

La QFT può essere vista come una trasformazione che sposta l'informazione dalla **base computazionale** (base z) alla **base di Fourier** (base delle fasi).

1.6.1 Sovrapposizione di tutti gli stati

Matematicamente, la QFT trasforma uno stato della base computazionale $|x\rangle$ in una sovrapposizione uniforme di tutti gli N stati:

$$\text{QFT} |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{i2\pi \frac{xk}{N}} |k\rangle$$

Tutti gli stati $|k\rangle$ hanno la stessa ampiezza $\frac{1}{\sqrt{N}}$, ma sono distinti da fattori di fase $e^{i2\pi \frac{xk}{N}}$. Di conseguenza, una misura immediata nella base computazionale produce una **distribuzione uniforme** (nel caso ideale).

1.6.2 Conteggio nella base di Fourier

Nella **base computazionale**, dove i bit classici sono mappati nei ket $|0\rangle$ e $|1\rangle$, i numeri interi sono rappresentati in formato binario. In questa configurazione, i qubit cambiano il proprio stato con frequenze diverse al crescere del valore decimale x :

- il bit meno significativo (*Least Significant Bit*, LSB) alterna il proprio stato ad ogni incremento unitario di x ;
- il bit successivo cambia stato ogni 2 incrementi;
- il bit più significativo (*Most Significant Bit*, MSB) è quello che varia più lentamente, cambiando stato solo dopo 2^{n-1} incrementi.

Al contrario, nella **base di Fourier** $|\tilde{x}\rangle$, l'informazione è codificata interamente nelle fasi relative dei qubit lungo il piano xy della sfera di Bloch. Le proprietà fondamentali di questa rappresentazione, coerentemente con la natura della trasformata, invertono la gerarchia di variazione:

- lo stato $|\tilde{0}\rangle$ corrisponde alla sovrapposizione uniforme $|+\rangle^{\otimes n}$, in cui tutti i qubit presentano una fase nulla;
- ogni valore decimale x viene mappato in un set univoco di rotazioni angolari (fasi) per ciascun qubit;
- la “velocità” di rotazione della fase è massima per il qubit che rappresenta la componente di Fourier più alta (MSB).

Questo comportamento permette di interpretare il registro come un “orologio quantistico”, dove le posizioni angolari indicano il valore codificato.

Analiticamente, la **fase accumulata** da ciascun qubit j nel piano xy (assumendo l'ordine standard post-SWAP) è determinata dalla relazione:

$$\phi_j = \frac{2\pi x}{2^{n-j}} \pmod{2\pi}, \quad \text{per } j = 0, 1, \dots, n-1,$$

dove n è il numero totale di qubit e $j = n-1$ identifica il MSB.

A titolo esemplificativo, si consideri un sistema di 4 qubit. Vediamo prima di tutto la rappresentazione dello stato $x = 0$:

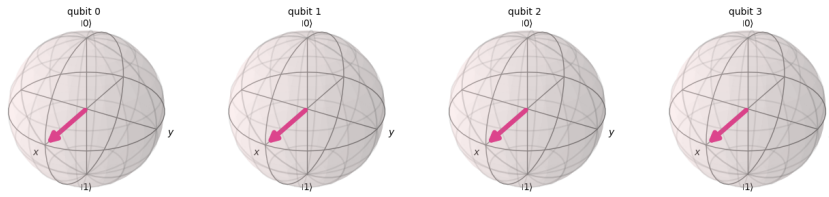


Figura 1.1: Rappresentazione sulle sfere di Bloch dello stato $|\tilde{0}\rangle = \text{QFT}|0\rangle$.

Incrementando il valore a $x = 1$, si ottiene lo stato trasformato $|\tilde{1}\rangle$, dove emerge chiaramente la diversa reattività dei componenti:

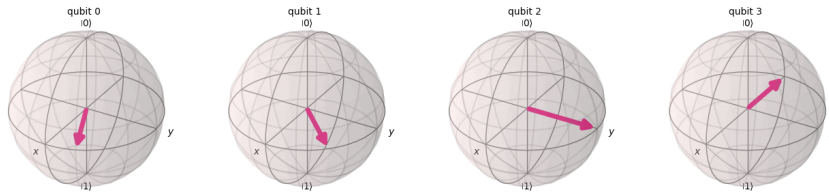


Figura 1.2: Rappresentazione sulle sfere di Bloch dello stato $|\tilde{1}\rangle = \text{QFT}|1\rangle$.

Dall'analisi delle sfere di Bloch si osserva come la componente di Fourier più alta coincida con il bit più significativo del registro di output:

- **Qubit 0 (LSB):** è caratterizzato dalla rotazione più **lenta**. La sua fase è $\phi_0 = \frac{2\pi x}{2^4} = \frac{\pi x}{8}$. Questo qubit richiede che x percorra l'intero spettro di valori ($2^4 = 16$) per completare una singola rotazione di 2π . Esso fornisce la **risoluzione macroscopica** del sistema, permettendo di distinguere la posizione del valore all'interno dell'intero ciclo.
- **Qubit 3 (MSB):** rappresenta la frequenza più alta e subisce le rotazioni di fase più **rapide**. La sua fase è $\phi_3 = \frac{2\pi x}{2^1} = \pi x$. Per ogni incremento unitario di x , questo qubit compie una rotazione di 180° (π radianti), agendo come indicatore della parità del valore x e definendo il limite superiore dello spettro di frequenze.

In sintesi, nella base di Fourier la gerarchia è ribaltata rispetto alla base computazionale: il MSB agisce come l'indicatore ad alta frequenza (variazione rapida), mentre il LSB definisce la struttura

angolare globale (variazione lenta), in accordo con la DFT classica. Questo è esattamente in accordo con l'espansione di:

$$\begin{aligned} QFT|x\rangle &= \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{xy} |y\rangle \\ &= \frac{1}{\sqrt{2^n}} \underbrace{\left(|0\rangle + e^{\frac{2\pi i}{2^1}x}|1\rangle\right)}_{\text{Fase MSB: } x \cdot \pi} \otimes \underbrace{\left(|0\rangle + e^{\frac{2\pi i}{2^2}x}|1\rangle\right)}_{\text{Fase: } x \cdot \frac{\pi}{2}} \otimes \cdots \otimes \underbrace{\left(|0\rangle + e^{\frac{2\pi i}{2^n}x}|1\rangle\right)}_{\text{Fase LSB: } x \cdot \frac{\pi}{2^n}} \end{aligned}$$

1.7 Implementazione circuitale

L'architettura del circuito QFT si basa sull'applicazione sequenziale di due tipologie di operatori:

- **Porte di Hadamard (H):** Utilizzate per creare sovrapposizione. Quando applicate al qubit j , esse inizializzano la fase basandosi sul bit x_j .
- **Porte di Rotazione Controllata (CP o CR_k):** Queste porte introducono le fasi frazionarie necessarie per la precisione della trasformata. Una porta di rotazione a singolo qubit è definita come:

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\frac{2\pi}{2^k}} \end{bmatrix}$$

Nel circuito, R_k viene applicata in modo condizionato. La porta **Controlled-Phase** $CP(\theta)$ agisce su due qubit (controllo e target) secondo la matrice:

$$CP(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{bmatrix}$$

Struttura del circuito: L'algoritmo procede processando i qubit uno alla volta. Per il qubit j , si applica prima una porta H , seguita da una serie di rotazioni CP controllate dai qubit $j+1, j+2, \dots, n$. Poiché l'ordine dei qubit risultante dalla decomposizione prodotto è invertito, il circuito si conclude tipicamente con una serie di porte **SWAP** per ripristinare l'ordine corretto dei bit (da MSB a LSB).

1.8 Vantaggi e Limiti Computazionali

La QFT è uno degli esempi più lampanti di **vantaggio quantistico**:

- **Efficienza:** Mentre l'algoritmo classico Fast Fourier Transform (FFT) richiede circa $O(N \log N)$ operazioni (dove $N = 2^n$), la QFT richiede solo $O(n^2)$ porte quantistiche. Se consideriamo la taglia dell'input n , il passaggio è da un costo esponenziale a uno **polinomiale**.
- **Applicazioni:** È il motore computazionale dell'**algoritmo di Shor** (per il logaritmo discreto e la fattorizzazione), della **Quantum Phase Estimation** (fondamentale per la chimica quantistica) e di numerosi algoritmi di **period-finding**.

Sfide tecnologiche: Nonostante il vantaggio teorico, l'implementazione su hardware reale (dispositivi NISQ) presenta delle criticità:

1. **Decoerenza:** Poiché la QFT richiede molte porte a due qubit (specialmente per n elevato), il tempo di esecuzione può superare il tempo di coerenza dei qubit.

2. **Precisione delle rotazioni:** Le porte R_k con k grande applicano rotazioni estremamente piccole ($2\pi/2^k$). Spesso queste rotazioni sono inferiori alla soglia di rumore del dispositivo.
3. **QFT Approssimata:** Per mitigare questi problemi, si utilizza spesso la *Approximate QFT*, che consiste nel troncare le rotazioni R_k quando $k > m$ (con $m \approx \log n$). Ciò riduce la profondità del circuito senza degradare significativamente la fedeltà del risultato per molte applicazioni pratiche (lo vedremo meglio in seguito).

Capitolo 2

Piattaforma

2.1 Accesso alla piattaforma IBM Quantum

L'implementazione sperimentale è stata realizzata utilizzando la piattaforma **IBM Quantum Cloud**, che consente l'accesso sia a simulatori classici sia a dispositivi quantistici reali (QPU).

L'accesso alla piattaforma è gratuito e avviene tramite registrazione con un account IBM. In particolare, è stato utilizzato il programma per studenti fornito da **IBM SkillsBuild**, che consente l'attivazione dell'account senza necessità di inserire una carta di credito.

Successivamente alla registrazione, è stata generata una chiave API personale attraverso il portale ufficiale: <https://quantum.cloud.ibm.com>. Questa chiave consente l'autenticazione al servizio e l'accesso ai backend quantistici disponibili. Il piano gratuito attualmente offre un limite di utilizzo pari a circa **10 minuti di computazione mensile su hardware quantistico reale**, soggetto alla disponibilità delle code di esecuzione.

2.2 Ambiente di sviluppo e strumenti utilizzati

Per lo sviluppo e l'esecuzione degli esperimenti è stato utilizzato l'ambiente **PyCharm**, configurato con supporto a **Jupyter Notebook**, al fine di organizzare in modo modulare l'implementazione degli algoritmi, le simulazioni e l'analisi dei risultati. L'uso dei notebook ha permesso di integrare in un unico ambiente:

- implementazione dell'algoritmo quantistico (QFT);
- simulazioni classiche;
- esecuzione su hardware quantistico;
- visualizzazione dei risultati e dei grafici.

2.3 Librerie e framework

L'implementazione si basa principalmente sul framework **Qiskit**, una libreria open-source sviluppata da IBM per la programmazione di circuiti quantistici, la simulazione e l'esecuzione su hardware quantistico reale e simulato. In particolare, sono stati utilizzati i seguenti moduli:

- **qiskit**: costruzione dei circuiti quantistici;
- **qiskit_aer**: simulazione classica su CPU;
- **qiskit_ibm_runtime**: accesso ai dispositivi quantistici reali.

Il simulatore **Aer** consente di eseguire circuiti quantistici in ambiente classico, offrendo diverse modalità di simulazione (ad esempio vettore di stato o campionamento). Esso permette di analizzare il comportamento del circuito in assenza di rumore oppure includendo modelli di errore realistici, risultando particolarmente utile per il debugging e la validazione degli algoritmi.

Inoltre, Qiskit mette a disposizione strumenti per l'ispezione e l'analisi dei circuiti, tra cui la visualizzazione grafica, il conteggio delle porte, la profondità del circuito e la misura delle prestazioni in termini di tempo di esecuzione e distribuzione delle probabilità in uscita.

2.4 Linguaggio di programmazione

L'intero progetto è stato sviluppato in **Python**, scelto per la sua integrazione nativa con Qiskit e per la disponibilità di librerie scientifiche dedicate all'analisi numerica e alla visualizzazione dei dati, come `matplotlib`.

Capitolo 3

Implementazione

3.1 Trasformata di Fourier Quantistica

3.1.1 Struttura dell'algoritmo

La **QFT** è realizzata come una sequenza di operazioni elementari su singoli qubit e coppie di qubit. In particolare, l'implementazione si basa su:

- porte di Hadamard (H), che generano sovrapposizioni;
- rotazioni di fase controllate ($CP(\theta)$), che introducono fasi relative tra gli stati;
- operazioni di swap finali, necessarie per correggere l'ordine dei qubit.

L'algoritmo segue uno *schema iterativo*: per ciascun qubit si applica una porta di Hadamard, seguita da una sequenza di rotazioni di fase controllate con tutti i qubit successivi. Al termine, si esegue una permutazione dell'ordine dei qubit tramite swap.

3.1.2 Pseudocodice

```
QFT(n qubits):  
    for i = n-1 down to 0:  
        apply H on qubit i  
        for j = i-1 down to 0:  
            apply  $CP(\pi / 2^{(i-j)})$  between qubits j and i  
  
    for i = 0 to n/2:  
        swap qubit i with qubit (n-i-1)
```

3.1.3 Implementazione in Python

```
def qft(qc_qft, n_qubits):  
    for i in reversed(range(n_qubits)):  
        qc_qft.h(i)  
  
        for j in reversed(range(i)):  
            angle = np.pi / (2 ** (i - j))  
  
            qc_qft.cp(angle, j, i)  
  
    for i in range(n_qubits // 2):  
        qc_qft.swap(i, n_qubits - i - 1)
```

L'implementazione segue direttamente la decomposizione circuitale della QFT. Il ciclo esterno itera sui qubit, applicando una porta di Hadamard che crea una sovrapposizione uniforme. Per ciascun qubit i , il ciclo interno introduce rotazioni di fase controllate con i qubit $j > i$, con angolo:

$$\theta = \frac{\pi}{2^{(j-i)}}$$

Tali rotazioni codificano l'informazione di frequenza nelle fasi relative degli stati quantistici. Infine, le operazioni di swap ripristinano l'ordine standard dei qubit, invertito dalla costruzione della QFT.

Questa decomposizione consente di esprimere la trasformazione unitaria della QFT come una sequenza di porte elementari, rendendola eseguibile sia su simulatori che su dispositivi quantistici reali.

Il circuito per la QFT su 4 qubit è:

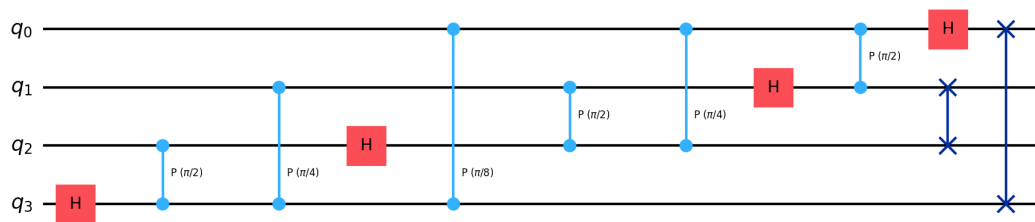


Figura 3.1: Circuito della QFT su 4 qubit.

3.2 Antitrasformata di Fourier Quantistica

L'**antitrasformata di Fourier quantistica** (IQFT, o QFT^{-1}) è l'operatore inverso della QFT e consente di ricostruire lo stato quantistico originale a partire dalla sua rappresentazione nel dominio delle frequenze. Essendo la QFT un operatore unitario, la sua inversa coincide con l'aggiunto, ed è ottenuta invertendo l'ordine delle operazioni e cambiando il segno delle fasi.

3.2.1 Struttura dell'algoritmo

L'algoritmo della IQFT segue la struttura inversa della QFT e si articola nei seguenti passaggi:

1. applicazione degli swap per invertire l'ordine dei qubit;
2. applicazione delle rotazioni di fase controllate con angolo opposto;
3. applicazione delle porte di Hadamard.

3.2.2 Pseudocodice

```

IQFT(n qubits):
  for i = 0 to n/2:
    swap qubit i with qubit (n-i-1)

  for i = 0 to n-1:
    for j = 0 to i-1:
      apply CP(-pi / 2^(i-j)) between qubits j and i
    apply H on qubit i

```

3.2.3 Implementazione in Python

```
def inverse_qft(qc_ift, n_qubits):
    for i in range(n_qubits // 2):
        qc_ift.swap(i, n_qubits - i - 1)

    for i in range(n_qubits):
        for j in range(i):
            angle = -np.pi / (2 ** (i - j))

            qc_ift.cp(angle, j, i)

    qc_ift.h(i)
```

L'implementazione riflette direttamente la struttura inversa della QFT. Gli swap iniziali correggono l'ordine dei qubit, mentre le rotazioni controllate vengono applicate con segno opposto per annullare le fasi introdotte nella trasformata diretta. Infine, le porte di Hadamard eliminano la sovrapposizione, riportando il sistema nella base computazionale.

Questa costruzione garantisce che la composizione $QFT^{-1} \circ QFT$ coincida con l'identità (come verificheremo in seguito negli esperimenti), permettendo di recuperare esattamente lo stato iniziale in assenza di errori.

Il circuito della IQFT su 4 qubit è mostrato di seguito:

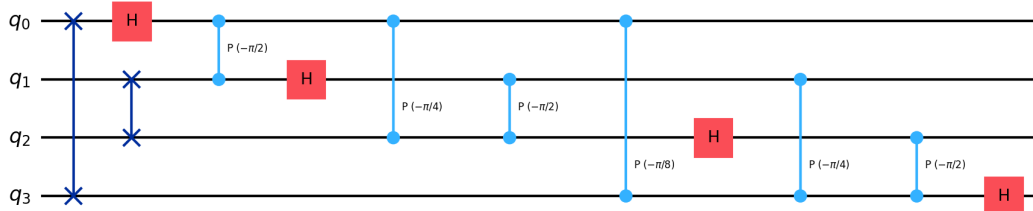


Figura 3.2: Circuito della QFT^{-1} su 4 qubit.

3.3 Trasformata di Fourier Quantistica Approssimata

La **QFT approssimata** è una variante della trasformata di Fourier quantistica in cui le rotazioni di fase con angolo molto piccolo vengono trascurate. Questa semplificazione consente di ridurre il numero di porte e la profondità del circuito, introducendo un errore controllato.

Poiché la QFT è composta da rotazioni di fase con angoli della forma:

$$\theta = \frac{\pi}{2^k},$$

i contributi associati a valori elevati di k risultano progressivamente meno significativi. È quindi possibile omettere tali operazioni senza alterare in modo rilevante il risultato finale.

3.3.1 Struttura dell'algoritmo

L'algoritmo ricalca quello della QFT standard, introducendo un criterio di selezione sulle rotazioni:

- applicazione delle porte di Hadamard su ciascun qubit;
- applicazione selettiva delle rotazioni di fase controllate;
- swap finali per correggere l'ordine dei qubit.

3.3.2 Pseudocodice

```

QFT_approx(n qubits, threshold):
    for i = n-1 down to 0:
        apply H on qubit i
        for j = i-1 down to 0:
            angle =  $\pi / 2^{(i-j)}$ 
            if angle > threshold:
                apply CP(angle) between qubits j and i

    for i = 0 to n/2:
        swap qubit i with qubit (n-i-1)

```

3.3.3 Implementazione in Python

```

def qft_approx(qc_approx_qft, n_qubits, threshold=0.1):
    for i in reversed(range(n_qubits)):
        qc_approx_qft.h(i)

        for j in reversed(range(i)):
            angle = np.pi / (2 ** (i - j))

            if angle > threshold:
                qc_approx_qft.cp(angle, j, i)

    for i in range(n_qubits // 2):
        qc_approx_qft.swap(i, n_qubits - i - 1)

```

L'implementazione introduce una soglia (`threshold`) che determina quali rotazioni mantenere. Le porte con angolo inferiore alla soglia vengono omesse, riducendo il numero totale di operazioni. Questo approccio consente un compromesso tra accuratezza e costo computazionale.

In particolare:

- le porte di Hadamard vengono sempre applicate;
- le rotazioni di fase vengono filtrate in base alla loro ampiezza;
- gli swap finali preservano la corretta disposizione dei qubit.

Questa variante è particolarmente rilevante nel contesto dei dispositivi NISQ, dove la riduzione della profondità del circuito contribuisce a mitigare gli effetti del rumore e della decoerenza.

Capitolo 4

Esperimenti e Risultati

In questa sezione vengono presentati gli **esperimenti** condotti al fine di valutare la correttezza e le prestazioni dell'implementazione della Trasformata di Fourier Quantistica (QFT), della sua inversa e delle relative varianti approssimate e rumorose. Le analisi sono state effettuate sia tramite simulazioni su CPU, sia mediante esecuzione su dispositivi quantistici reali (QPU).

L'indagine comprende verifiche di consistenza logica, confronti tra comportamento ideale e in presenza di rumore, nonché la valutazione della complessità del circuito e dei tempi di esecuzione.

Ciascun esperimento è stato eseguito in tre diverse modalità:

- simulazione ideale tramite **AerSimulator**;
- simulazione con rumore, mediante un modello realistico;
- esecuzione su dispositivo quantistico reale (QPU).

Tutti i test sono stati condotti su circuiti a $n = 3$ qubit, mantenendo invariata tale configurazione in ciascuno degli scenari considerati. Inoltre, per garantire la confrontabilità dei risultati, si è utilizzato in tutti i casi lo stesso stato iniziale:

$$|\psi\rangle = |010\rangle.$$

Tale stato appartiene alla base computazionale ed è rappresentato nella figura che segue:

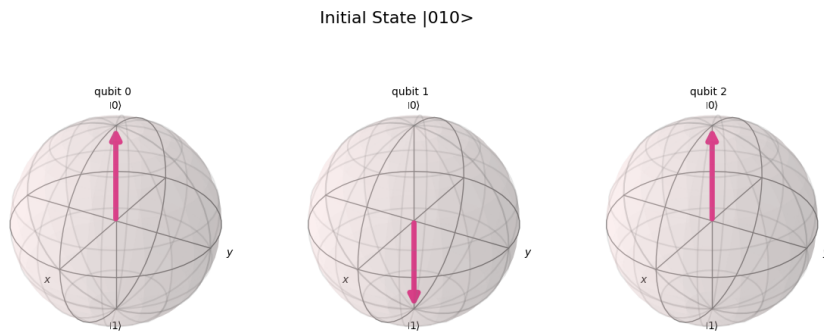


Figura 4.1: Rappresentazione dello stato $|010\rangle$ nella base computazionale.

Lo stato $|010\rangle$ viene preparato a partire dallo stato iniziale $|000\rangle$ mediante l'applicazione dell'operatore

$$I \otimes X \otimes I,$$

ossia applicando una porta di Pauli- X al secondo qubit.

Il semplice circuito di preparazione dello stato è:

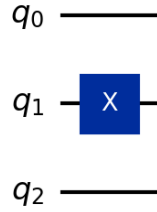


Figura 4.2: Circuito di preparazione dello stato $|010\rangle$.

4.1 Test di correttezza

Il primo esperimento verifica la correttezza dell'implementazione della QFT e della sua inversa. In base alla teoria, la loro composizione deve restituire lo stato iniziale:

$$\text{QFT}^{-1}(\text{QFT}(|\psi\rangle)) = |\psi\rangle$$

Il circuito prepara lo stato $|010\rangle$, applica la QFT seguita dalla sua inversa e infine misura il sistema. In assenza di errori, la distribuzione finale deve essere concentrata sullo stato iniziale.

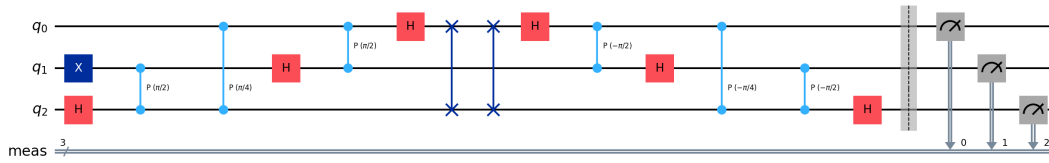


Figura 4.3: Correttezza della QFT

Analisi del risultato L'output osservato è:

Soundness test (should recover $|010\rangle$): {'010': 1024}

Tutte le esecuzioni restituiscono lo stato iniziale, con:

$$P(010) = 1, \quad P(x) = 0 \quad \forall x \neq 010$$

Il risultato conferma la correttezza dell'implementazione e l'assenza di errori numerici apprezzabili nella simulazione ideale, in accordo con la proprietà di unitarietà:

$$U_{QFT}^\dagger U_{QFT} = I$$

dove U_{QFT} è l'operatore associato alla QFT.

4.2 Trasformata di Fourier su Simulatore Aer

4.2.1 Esecuzione della QFT

Si applica la **Trasformata di Fourier Quantistica (QFT)** allo stato iniziale $|010\rangle$ (che rappresenta il valore decimale $x = 2$), preparato come descritto in precedenza.

La QFT mappa gli stati dalla base computazionale alla **base di Fourier**, redistribuendo l'informazione del valore x nelle fasi relative dei singoli qubit. Il risultato della trasformazione può essere visualizzato efficacemente tramite le sfere di Bloch.

Nel caso $x = 2$, l'informazione non è più localizzata, ma codificata nelle fasi dei componenti del registro. La rappresentazione è:

$$\phi_j = \frac{2\pi x}{2^{n-j}} \pmod{2\pi}, \quad \text{per } j = 0, 1, \dots, n-1,$$

dove n è il numero totale di qubit e $j = 0$ identifica il **LSB**, mentre $j = n-1$ identifica il **MSB**. Per $x = 2$ otteniamo:

- **Qubit 0 (LSB):** La rotazione di fase generale è definita da $\phi_0 = \frac{2\pi x}{2^n}$. Sostituendo $x = 2$:

$$\phi_0 = \frac{2\pi \cdot 2}{2^{3-0}} = \frac{4\pi}{8} = \frac{1\pi}{4}$$

Il bit meno significativo subisce la rotazione più lenta (frequenza più bassa), accumulando una fase piccola e risultando nello stato $|+i\rangle$.

- **Qubit 1:** La rotazione di fase generale è definita da $\phi_1 = \frac{2\pi x}{2^{n-1}}$. Sostituendo $x = 2$:

$$\phi_1 = \frac{4\pi}{2^{3-1}} = \frac{4\pi}{4} = \pi$$

La fase è maggiore rispetto al qubit precedente, riflettendo una frequenza più alta, risultando nello stato $|-\rangle$.

- **Qubit 2 (MSB):** La rotazione di fase generale è definita da $\phi_2 = \frac{2\pi x}{2^{3-2}}$. Sostituendo $x = 2$:

$$\phi_3 = \frac{4\pi}{2^{3-2}} = \frac{4\pi}{2} = 2\pi$$

Il bit più significativo ha la frequenza di rotazione più alta, completando un giro completo e risultando nello stato $|+\rangle$.

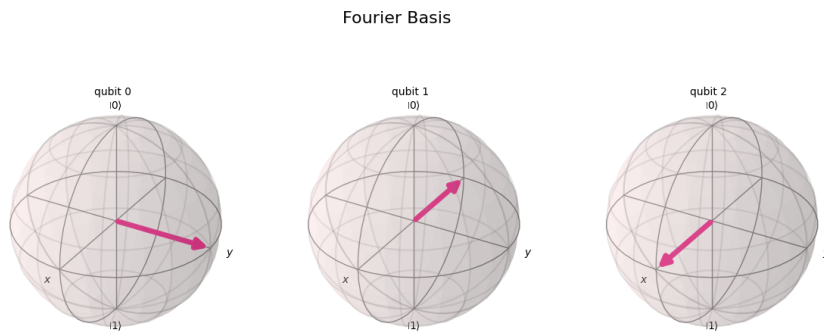


Figura 4.4: Rappresentazione dello stato trasformato dalla QFT sulle sfere di Bloch.

Segue la fase di misura. Poiché le ampiezze differiscono unicamente per un fattore di fase complesso $e^{i\phi_k}$, il modulo quadro non dipende dalla fase. Ne risulta una distribuzione di probabilità uniforme sulla base computazionale:

$$P(k) = \left| \frac{1}{\sqrt{2^n}} e^{i\phi_k} \right|^2 = \frac{1}{2^n}, \quad \forall k \in \{0, \dots, 2^n - 1\}.$$

Nel caso specifico di $n = 3$:

$$P(k) = \frac{1}{8}, \quad \forall k \in \{0, \dots, 7\}.$$

Il circuito implementato, comprensivo di porte di Hadamard, rotazioni controllate R_k e swap finali per invertire l'ordine dei qubit, è mostrato nella figura seguente:

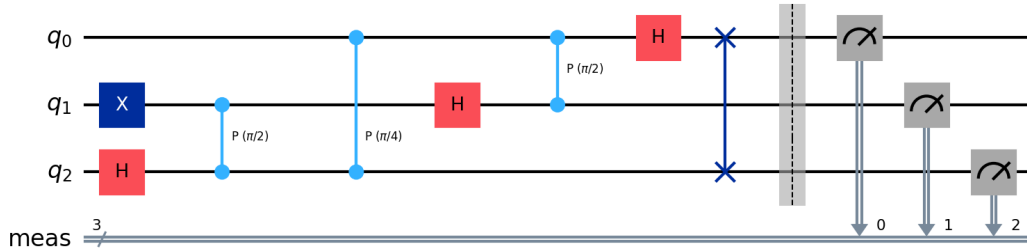


Figura 4.5: Circuito della QFT su 3 qubit con swap finali.

4.3 Comparazione tra simulazione ideale e rumorosa

Il passo successivo consiste nell'analizzare il comportamento del circuito confrontando la simulazione ideale con quella in presenza di rumore statistico. I simulatori quantistici eseguono il circuito per un numero elevato di iterazioni (*shots*) al fine di ricostruire la distribuzione di probabilità degli stati misurati. Poiché la misura quantistica è un processo intrinsecamente probabilistico, una singola esecuzione non è sufficiente a caratterizzare lo stato finale del sistema.

L'errore statistico sulle frequenze osservate decade secondo la legge:

$$\epsilon \approx \mathcal{O}\left(\frac{1}{\sqrt{N}}\right),$$

dove N rappresenta il numero di *shot*.

4.3.1 Analisi dei risultati

State	Ideal Counts	Noisy Counts	Ideal Prob	Noisy Prob	Delta (Abs)
$ 000\rangle$	141	111	0.1377	0.1084	0.0293
$ 001\rangle$	122	129	0.1191	0.1260	0.0069
$ 010\rangle$	115	113	0.1123	0.1104	0.0019
$ 011\rangle$	123	148	0.1201	0.1445	0.0244
$ 100\rangle$	133	144	0.1299	0.1406	0.0107
$ 101\rangle$	133	127	0.1299	0.1240	0.0059
$ 110\rangle$	123	123	0.1201	0.1201	0.0000
$ 111\rangle$	134	129	0.1309	0.1260	0.0049

I dati in tabella evidenziano come anche la simulazione ideale presenti lievi scostamenti rispetto alla distribuzione teorica uniforme, un fenomeno imputabile esclusivamente al campionamento

Monte Carlo su un numero finito di campioni. Parallelamente, la simulazione rumorosa introduce ulteriori fluttuazioni, pur mantenendo un profilo statistico coerente con il caso ideale.

Analisi globale L'errore complessivo tra le due distribuzioni, quantificato mediante la distanza L_1 , risulta:

$$\text{Total } L_1 \text{ Distance (Error)} = 0.0420$$

4.4 Libreria vs implementazione custom

In questa sezione vengono messe a confronto le prestazioni dell'implementazione personalizzata della QFT con le routine fornite dalla libreria standard (Qiskit), analizzando le rispettive distribuzioni di probabilità.

4.4.1 Analisi dei risultati

State	Ideal Counts	Library Counts	Ideal Prob	Library Prob	Delta (Abs)
$ 000\rangle$	141	116	0.1377	0.1133	0.0244
$ 001\rangle$	122	125	0.1191	0.1221	0.0029
$ 010\rangle$	115	149	0.1123	0.1455	0.0332
$ 011\rangle$	123	124	0.1201	0.1211	0.0010
$ 100\rangle$	133	144	0.1299	0.1406	0.0107
$ 101\rangle$	133	133	0.1299	0.1299	0.0000
$ 110\rangle$	123	91	0.1201	0.0889	0.0312
$ 111\rangle$	134	142	0.1309	0.1387	0.0078

Analisi globale L'errore complessivo tra le due distribuzioni (distanza L_1) è pari a:

$$\text{Total } L_1 \text{ Distance (Error)} = 0.0557$$

Il valore ottenuto indica una solida corrispondenza tra le due implementazioni. Le deviazioni residue sono attribuibili agli effetti del campionamento stocastico e alla differente sensibilità del circuito alle micro-variazioni nella decomposizione dei gate elementari.

4.5 Approssimata vs ideale

Viene qui valutato l'impatto di una soglia di approssimazione impostata a `threshold = 0.2`.

4.5.1 Analisi dei risultati

State	Ideal Counts	Approx Counts	Ideal Prob	Approx Prob	Delta (Abs)
$ 000\rangle$	141	157	0.1377	0.1533	0.0156
$ 001\rangle$	122	124	0.1191	0.1211	0.0020
$ 010\rangle$	115	122	0.1123	0.1191	0.0068
$ 011\rangle$	123	126	0.1201	0.1230	0.0029
$ 100\rangle$	133	134	0.1299	0.1309	0.0010
$ 101\rangle$	133	130	0.1299	0.1270	0.0029
$ 110\rangle$	123	115	0.1201	0.1123	0.0078
$ 111\rangle$	134	116	0.1309	0.1133	0.0176

Analisi globale L'errore complessivo, misurato tramite distanza L_1 , risulta:

$$\text{Total } L_1 \text{ Distance (Error)} = 0.0566$$

L'analisi conferma che l'approssimazione mantiene un'elevata fedeltà rispetto al caso ideale. Le deviazioni introdotte sono limitate e si concentrano principalmente sugli stati i cui contributi di fase risultano più sensibili alla troncatura delle rotazioni controllate.

4.6 Esecuzione su QPU

Vediamo ora il comportamento della trasformata di Fourier quando viene eseguita su una QPU reale, analizzando le deviazioni rispetto all'implementazione ideale dovute agli errori hardware.

4.6.1 Analisi dei risultati

State	Ideal Counts	QPU Counts	Ideal Prob	QPU Prob	Delta (Abs)
$ 000\rangle$	141	120	0.1377	0.1172	0.0205
$ 001\rangle$	122	139	0.1191	0.1357	0.0166
$ 010\rangle$	115	137	0.1123	0.1338	0.0215
$ 011\rangle$	123	145	0.1201	0.1416	0.0215
$ 100\rangle$	133	115	0.1299	0.1123	0.0176
$ 101\rangle$	133	113	0.1299	0.1104	0.0195
$ 110\rangle$	123	115	0.1201	0.1123	0.0078
$ 111\rangle$	134	140	0.1309	0.1367	0.0059

Analisi globale L'errore complessivo tra simulazione ideale e hardware reale è pari a:

$$\text{Total } L_1 \text{ Distance (Error)} = 0.0654$$

Questo risultato riflette l'effetto combinato del rumore di decoerenza, degli errori nei gate logici e degli errori di lettura (*readout error*). Tali fattori introducono deviazioni sistematiche che, pur preservando la struttura globale della distribuzione, comportano una degradazione della qualità dei dati più marcata rispetto ai modelli rumorosi simulati.

4.7 Confronto sinottico degli errori

In questa sezione si riassumono gli errori globali (distanza L_1) riscontrati nei diversi scenari sperimentali rispetto al riferimento ideale.

Caso	Total L1 Distance (Error)
HW (simulazione rumorosa)	0.0420
Approximated QFT	0.0557
Library QFT	0.0566
QPU (hardware reale)	0.0654

4.8 Analisi delle Performance: Tempi di Esecuzione e Risorse Hardware

In questa sezione viene analizzata l'efficienza computazionale del circuito proposto, confrontando le prestazioni della simulazione classica su CPU rispetto all'esecuzione su hardware quantistico reale (QPU). L'analisi prende in esame non solo il tempo di calcolo, ma anche l'impatto sulla memoria di sistema.

4.8.1 Simulazione su CPU: Il Limite della Complessità Esponenziale

I test condotti in ambiente CPU mostrano il tipico limite dei simulatori a stato vettoriale (*statevector simulators*).

Numero di Qubit	15	20	25	30
Tempo (s)	0.151	0.205	2.691	77.287
RAM Stimata	~512 KB	~16 MB	~512 MB	~16 GB

Analisi della Complessità: Il tempo di esecuzione su CPU segue una crescita esponenziale $O(2^n)$. Questo accade perché simulare un sistema quantistico significa manipolare un vettore di ampiezze di probabilità di dimensione 2^n nel campo complesso.

Gestione della RAM: Oltre al tempo, la memoria RAM rappresenta il vero "collo di bottiglia" fisico. Ogni qubit aggiunto raddoppia lo spazio di memoria richiesto:

- Per $n = 30$ qubit, lo stato è rappresentato da 2^{30} numeri complessi. Utilizzando una precisione a 128 bit (doppia precisione), sono necessari circa **16 GB di RAM**.
- Il salto verso i 35-40 qubit richiederebbe hardware di classe enterprise (Terabyte di RAM), rendendo la simulazione locale impraticabile.

4.8.2 Esecuzione su QPU: Overhead Infrastrutturale

I dati ottenuti tramite il backend `ibm_fez` evidenziano un comportamento radicalmente diverso, dove il tempo di esecuzione appare quasi indipendente dal numero di qubit.

Numero di Qubit	15	20	25	30
Tempo Totale (s)	6.0	7.0	7.0	7.0

Il concetto di Latenza vs Tempo di Gate: Il tempo riportato non rappresenta la durata delle operazioni logiche sui qubit (che avviene nell'ordine dei microsecondi), bensì l'intero *workflow* di gestione del job:

1. **Transpilation:** Ottimizzazione del circuito logico per l'architettura fisica specifica.
2. **Queueing:** Tempo di attesa nelle code del server IBM Quantum.
3. **Calibration & Latency:** Tempo di invio dei segnali a microonde criogenici e recupero dei dati dai readout.

In questo scenario, l'incremento di qubit da 15 a 30 ha un impatto trascurabile sul tempo totale, poiché l'overhead di comunicazione domina la componente computazionale.

4.8.3 Confronto e Considerazioni Finali

Il vantaggio quantistico inizia a manifestarsi visivamente nel punto di incrocio tra la curva esponenziale della CPU e la retta (quasi) costante della QPU.

Tabella 4.1: Rapporto di efficienza tra i due approcci.

Qubits	Tempo CPU (s)	Tempo QPU (s)	Ratio (QPU/CPU)
15	0.151	6.00	39.66 (Svantaggio)
20	0.205	7.00	34.13
25	2.691	7.00	2.60
30	77.287	7.00	0.09 (Vantaggio)

Graficamente:

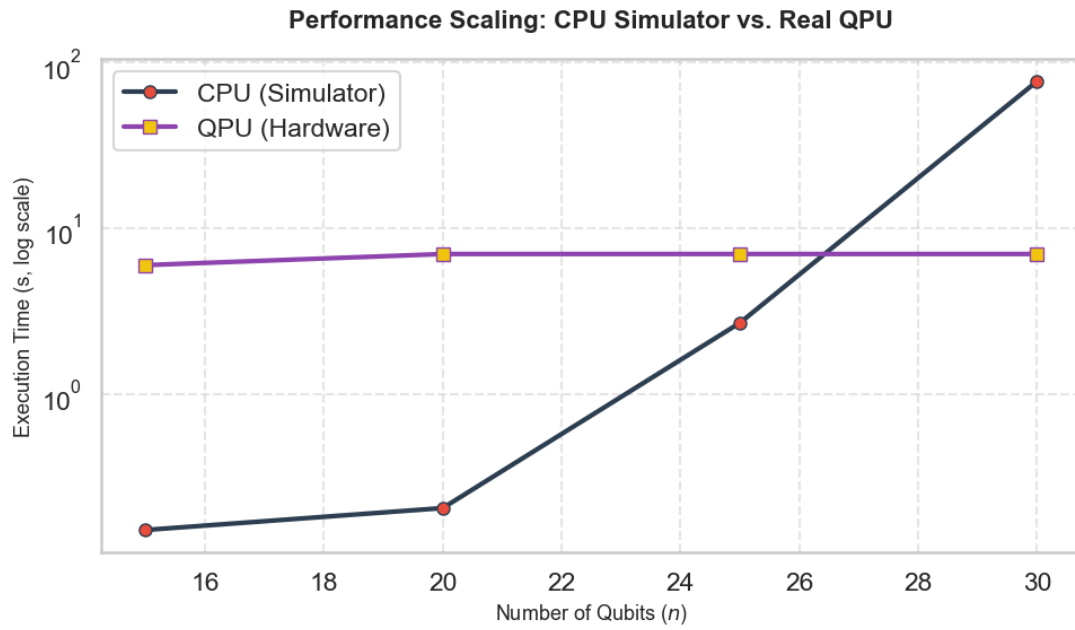


Figura 4.6: Rappresentazione dello stato trasformato dalla QFT sulle sfere di Bloch.

Conclusioni: Mentre per circuiti piccoli la simulazione classica è nettamente superiore grazie all'assenza di latenza di rete, per $n \geq 30$ la QPU diventa la scelta obbligata. Oltre questa soglia, la CPU non solo diventa lenta, ma rischia il *crash* di sistema per saturazione della memoria RAM, mentre la QPU continua a operare in un regime di tempo costante, limitata solo dalla coerenza dei qubit e dalla fedeltà delle porte logiche.

Capitolo 5

Conclusioni

Il presente lavoro ha analizzato l'implementazione e le performance della Trasformata di Fourier Quantistica (QFT) tramite il framework Qiskit, mettendone a confronto le varianti manuali, di libreria e approssimate in scenari ideali e rumorosi.

L'analisi sperimentale ha confermato l'efficacia dell'operatore, validando l'invertibilità unitaria ($QFT^{-1} \circ QFT = I$) attraverso il recupero dello stato iniziale in un ambiente deterministico. Le evidenze principali emerse dai test includono:

- **Fedeltà e Rumore:** La simulazione rumorosa ha evidenziato una degradazione delle distribuzioni di probabilità proporzionale alla profondità del circuito. La metrica della distanza $L1$ ha confermato come l'accumulo di errori di gate e la decoerenza restino i principali ostacoli per le architetture NISQ.
- **Trade-off Accuratezza-Complessità:** La QFT approssimata si è dimostrata efficace nel mitigare la profondità circuitale; tuttavia, il risparmio in termini di errore hardware deve essere bilanciato con l'incremento dell'errore algoritmico intrinseco.
- **Ottimizzazione delle Risorse:** L'implementazione nativa di Qiskit ha garantito una gestione più efficiente della *gate depth* rispetto alla codifica manuale, ottimizzando il processo di *transpilation* per il backend specifico.

Il confronto prestazionale tra CPU e QPU ha messo in luce una dicotomia fondamentale. Sebbene la simulazione classica offra tempi di risposta rapidi per il range di qubit analizzato, essa è soggetta a un limite di memoria insuperabile dovuto alla crescita esponenziale del vettore di stato (2^n). Al contrario, il tempo di esecuzione su hardware reale è risultato dominato da **overhead infrastrutturali** (code di sistema, latenza cloud e calibrazione), rendendo il tempo di calcolo effettivo una frazione trascurabile del *turnaround time* totale.

Considerazioni finali Lo studio ha fornito una visione pragmatica del calcolo quantistico attuale: sebbene la teoria della QFT sia solida, la sua applicazione pratica richiede strategie avanzate di mitigazione dell'errore. In conclusione, mentre la simulazione classica rimane lo standard per il debugging e la prototipazione, l'impiego delle QPU si conferma indispensabile per caratterizzare il comportamento degli algoritmi in presenza di rumore fisico, segnando il passaggio cruciale dalla teoria alla validazione sperimentale.

Capitolo 6

Riferimenti e Risorse Online

- **Qiskit Documentation – Quantum Fourier Transform:** Guida ufficiale all'implementazione del circuito e API reference.
<https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.QFT>
- **Qiskit Textbook – QFT Notebook:** Repository ufficiale del corso interattivo con dettagli implementativi in Python.
<https://github.com/Qiskit/textbook/blob/main/notebooks/ch-algorithms/quantum-fourier-transform.ipynb>
- **IBM Quantum Learning – Computer Science QFT:** Modulo didattico avanzato sulla logica dell'algoritmo fornito da IBM.
<https://quantum.cloud.ibm.com/learning/en/modules/computer-science/qft>
- **Wikipedia – Quantum Fourier Transform:** Panoramica teorica, derivazione matematica e matriciale dell'operatore.
https://en.wikipedia.org/wiki/Quantum_Fourier_transform